

Routability-aware Packing for High-density Nonvolatile FPGAs

Huichuan Zheng

School of Computer Science and
Technology, Shandong University

Yuqing Xiong

School of Computer Science and
Technology, Shandong University

Jian Zuo

School of Computer Science and
Technology, Shandong University

Hao Zhang

School of Computer Science and
Technology, Shandong University

Zhenge Jia

School of Computer Science and
Technology, Shandong University

Mengying Zhao*

School of Computer Science and
Technology, Shandong University

Abstract—Nonvolatile field-programmable gate arrays (NVFPGAs) can use multi-level cell (MLC) nonvolatile memories (NVMs) to enhance their logic density. However, the high-density design of NVFPGAs degrades the intra-routability of configurable logic blocks (CLBs), which significantly prolongs the time consumed by the packing process in the computer-aided design (CAD) flow. To relieve the efficiency degradation, in this paper, we propose a routability-aware re-pair stage to adjust the logical-physical look-up table (LUT) assignments to mitigate the congestion and improve their intra-routability, thereby reducing the packing time. In addition, exploiting the structural equivalence of MLC LUTs, we remove unnecessary intra-routing attempts from packing to further improve efficiency. Evaluation shows the proposed strategies reduce packing time by 41.48% on average.

Index Terms—nonvolatile memory (NVM), multi-level cell (MLC), field-programmable gate array (FPGA), computer-aided design (CAD), packing.

I. INTRODUCTION

Field-programmable gate array (FPGA) is a promising option for many emerging applications, including near memory computing [1], [2], computer vision [3], [4], data encryption [5], [6], and large language model [7], [8], due to its high parallelism and reconfigurable flexibility. To deploy these applications, corresponding functions are transformed into small truth tables, which are stored in look-up tables (LUTs) inside configurable logic blocks (CLBs). With support of other programmable resources, i.e., routing resources and on-chip memories, the functionality can be implemented on FPGAs. As the complexity of applications grows, the demand for FPGA's capacity also continues to expand [9]–[12].

Traditional FPGAs adopt static random-access memory (SRAM) as their on-chip memory, which suffers from issues such as limited logic capacity and high leakage power. In contrast, emerging nonvolatile memories (NVMs) have advantages of high storage density, near-zero leakage power, and non-volatility. Thus, they are proposed to replace SRAM

to build nonvolatile FPGAs (NVFPGAs), e.g., phase change memory based NVFPGA [13]–[15], magnetoresistive RAM based NVFPGA [16], and Resistive RAM based NVFPGAs [17]–[19], where resources including logic [20], routing [21]–[23] as well as on-chip memories [24] can be designed with NVMs. These representative NVMs can also operate as multi-level cell (MLC), which further increases the capacity of NVFPGAs.

Although MLC NVM enables high-density design for NVFPGAs, it also introduces challenges for the FPGA computer-aided design (CAD). The CAD flow transforms a target circuit in Hardware Description Language (HDL) to the bitstreams that can run in FPGA platforms. As shown in Fig. 2 (a), a typical FPGA CAD flow contains the main processes of front-end synthesis, technology mapping, packing, placement, and routing [25]. Taking the packing process as an example, it clusters LUTs into CLBs, so that the subsequent placement and routing can be conducted at the granularity of CLBs instead of LUTs. A CLB is successfully packed if logical LUTs can be well paired with physical positions inside it, and these logics are routable with limited intra-CLB routing resources. Since MLC FPGA architecture enhances the logic density, the number of logical connections within a CLB also significantly increases, which degrades the intra-routability. Thus, the packing algorithm has to invoke the time-consuming intra-routing process more frequently to ensure the routability of the packing solution, which seriously affects the efficiency of the packing process. According to [26], the time consumed by the packing process for MLC NVFPGA is about $8.9\times$ higher than that for traditional SRAM-based FPGA.

Thus, in this paper, to relieve the degradation of intra-routability, we leverage routability considerations to guide the packing process, thereby reducing intra-routing invocations and improving packing efficiency. Specifically, we propose a routability-aware re-pair stage and integrate it into the existing packing process. To mitigate the intra-CLB congestion, it pairs logics from heavily congested regions to those regions with fewer logics. Through the proposed re-pair stage, some unroutable packing solutions could become routable so that a large number of intra-routing invocations could be avoided,

This work was supported by the National Key Research and Development Program of China (Grant No. 2022YFB4502004), the Natural Science Foundation of China (Grant No. 62372270), and Young Talent of Lifting engineering for Science and Technology in Shandong, China (Grant No. SDAST2024QTA078).

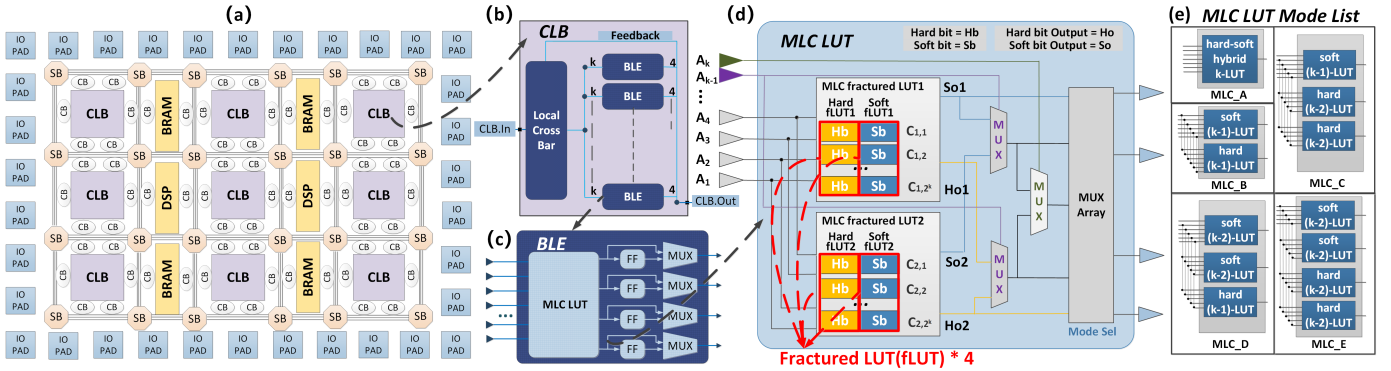


Fig. 1. The NVFPGA architecture proposed in [20]: (a) FPGA model, (b) CLB model, (c) BLE model, (d) MLC LUT model, and (e) MLC LUT mode list.

which can significantly improve the efficiency of the packing process for MLC NVFPGAs. In addition, exploiting the structural characteristics of MLC logic resources, we propose a duplicate intra-routing removal technique to avoid the routing attempts for input-equivalent packing solutions. The contributions of this paper are listed as follows.

- A routability-aware re-pair stage is proposed to improve the intra-routability of initially unroutable CLBs.
- Based on structural characteristics of MLC physical LUTs, an input-equivalent LUT removal technique is proposed to improve the efficiency of the packing process.
- Comprehensive evaluations are conducted on the proposed techniques to validate their effectiveness.

The remainder of this paper is organized as follows. Section II introduces the background and summarizes related work. Section III introduces the re-pair stage based method to optimize the packing efficiency. Section IV gives the evaluation results and discussions, followed by overhead analysis. Section V concludes this paper.

II. BACKGROUND AND RELATED WORK

A. MLC LUT in NVFPGA

A typical FPGA architecture includes configurable logic blocks (CLBs), switch boxes (SBs), connection boxes (CBs), and block random access memories (BRAMs), as shown in Fig. 1 (a). Among these components, CLBs are the main logic resources to implement combinational and sequential logics. As Fig. 1 (b) demonstrates, each CLB contains a cluster of basic logic elements (BLEs) along with a fully populated crossbar. As shown in Fig. 1 (c), a BLE can be further decomposed into one look-up table (LUT), some flip-flops (FFs), and several multiplexers (MUXs). Fig. 1 (d) shows the MLC k -LUT structure proposed in [20], whose logic capacity is doubled compared with the original LUT structure. Here k denotes the maximum number of inputs the LUT can support. An MLC LUT contains four fractured ($k - 2$)-LUTs, which are the smallest unit of logics in this structure. Fig. 1 (e) illustrates the mode list of the proposed MLC LUT structure. These fractured LUTs can be configured independently or work in group as a bigger LUT with more inputs, i.e., as ($k - 1$)-LUT or k -LUT. This structure significantly improves the logic density and flexibility of CLBs. As reported in [20],

compared with SRAM-based FPGAs, this MLC NVFPGA architecture achieves improvements in area, performance, and leakage power by 32.66%, 7.45%, and 96.13%, respectively.

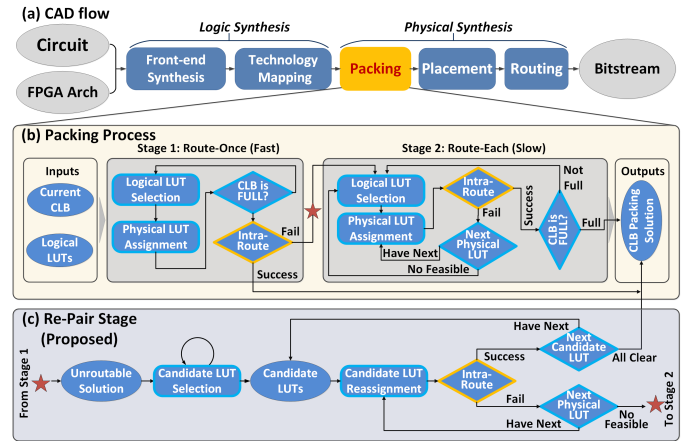


Fig. 2. The FPGA CAD flow, including: (a) the overview of a typical FPGA CAD flow, (b) the current packing process in the Verilog-To-Routing (VTR) flow [25], (c) the proposed re-pair stage.

B. Packing Process for FPGAs

The packing process clusters individual LUTs into CLBs, whose results will influence the subsequent processes of physical synthesis. A well-packed solution can significantly reduce the number of CLBs and the inter-CLB connection, greatly improving the efficiency of the placement and routing procedures. Currently, the representative packing algorithm adopted in Verilog-To-Routing (VTR) [25] contains two stages, denoted as Route-Once stage and Route-Each stage in this paper. As Fig. 2 (b) shows, in the Route-Once stage, LUTs are packed into the CLB until no more can be added. After these LUTs are packed, an intra-routing process is invoked to verify whether the packing solution is routable. If the current CLB is routable, the packing process for this CLB is finished. If it is unroutable, the packing process enters the Route-Each stage, where the routing process will be invoked after each time an LUT is added into the CLB. Since routing is usually the most time-consuming process in the CAD flow, the Route-Each stage takes significantly longer time than the Route-Once stage.

There have been many studies conducted on the packing process. Li et al. [27] propose a physical-and-congestion-aware packing algorithm, which achieves the co-optimization

in wirelength and routability. Elgammal and Betz [28] propose a re-clustering technique to improve the wirelength and critical path delay through dynamically updating the packed netlist during the packing and placement processes. Yu et al. [29] propose a quantitative rule to guide the packing process for neural networks, thereby improving the performance of the final FPGA design. Chen et al. [30] propose DP-Pack to perform packing in parallel. Rajarathnam et al. [31] propose DREAMPlaceFPGA-PL, which uses GPU to accelerate the packing process.

However, the above-mentioned works are designed for traditional SRAM-based FPGAs. Since NVFPGA introduces new characteristics, the CAD flow should also be modified to bridge the architectural differences and address the challenges. Liu et al. [26] propose a performance-aware packing algorithm for MLC NVFPGAs, which addresses the performance asymmetry of MLC NVM and reduces the critical path delay. However, it does not address the issue of packing efficiency degradation caused by MLC LUTs.

In this paper, focusing on the packing efficiency for MLC NVFPGAs, we propose a re-pair stage to reduce the number of invoking the time-consuming intra-routing process.

III. ROUTABILITY-AWARE RE-PAIR FOR PACKING OPTIMIZATION

In this section, we will introduce the proposed routability-aware re-pair based packing optimization method for MLC NVFPGAs in detail.

A. Motivation and Main Idea

To look into details of packing in MLC FPGAs, we conduct preliminary experiments on a series of representative test benches and find that the Route-Each stage occupies 91.16% of the total packing time. The statistics also show that 73.53% of CLBs cannot obtain routable solutions at the Route-Once stage, resulting in extensive invocations of the time-consuming Route-Each stage. Thus, the packing efficiency can be improved if the number of CLBs entering Route-Each is reduced.

When tracking packing solutions of CLBs, we found that some CLBs are unroutable at the Route-Once stage, but become routable after the Route-Each stage without changes in LUT selection. The difference lies in the logical-physical LUT assignments.

In the Route-Once stage, each LUT is assigned to the first physical LUT selected by the packing algorithm, resulting in imbalanced logic distribution and poor routability. On the contrary, in the Route-Each stage, these congested LUTs can be assigned to other physical LUTs if the previous attempts fail, as the routability can be checked immediately. Thus, the Route-Each stage keeps congestion-free throughout the stage.

These initially unroutable CLBs can become routable through adjusting the logical-physical LUT pairs, so that the extensive intra-routing of the Route-Each stage can be reduced. According to the statistical results, there are on average 30.30% of initially unroutable CLBs select the same LUTs

Algorithm 1 The Proposed Re-Pair Stage

Require: Congestion information from previous routing attempt C_{prev} ; List of LUTs packed in the previous stage.
Ensure: PACKING_STATUS

```

// Candidate LUT Selection
1: Initialize  $CPI$  for each LUT according to  $C_{prev}$ ;
2: Initialize  $N_{sel}$  based on size of available physical LUTs;
3: Initialize stack  $CLUT$  for selected candidate LUTs;
4: while  $N_{sel} > 0$  do
5:   Select  $LUT_{sel}$  from unmarked BLE with the max  $CPI$ ;
6:    $CLUT.push(LUT_{sel})$ ;
7:    $N_{sel} = N_{sel} - Sizeof(LUT_{sel})$ ;
8:   Mark the corresponding BLE;
9:   if all congested BLEs are marked then
10:     break;
11:   end if
12: end while
// Candidate LUT Reassignment
13: while ! $CLUT.empty()$  do
14:    $LUT_c = CLUT.pop()$ ;
15:   while  $LUT_c$  has feasible physical LUT do
16:     Select physical LUT  $P$  with the min  $RS$ ;
17:     if  $P$  is duplicate then
18:       continue;
19:     end if
20:     Assign  $LUT_c$  in  $P$ ;
21:     if  $Try\_route\_CLB()$  is TRUE then
22:       Record  $LUT_c$ 's new assignment;
23:       break;
24:     else if  $P$  is a  $(k - 2)$ -LUT then
25:       Set  $(k - 2)$ -LUTs within  $P$ 's BLE as duplicate;
26:     end if
27:   end while
28:   if reassigning  $LUT_c$  fails then
29:     PACKING_STATUS = FAIL;
30:     break;
31:   end if
32: end while
33: return PACKING_STATUS;

```

in the subsequent Route-Each stage, indicating a promising optimization potential.

The main idea of the proposed re-pair stage is to consider intra-routability and utilize unoccupied physical LUTs in the previous failed packing solution to mitigate intra-CLB congestion. Specifically, congested LUTs will be paired to BLEs with fewer logics so that routability can be improved. If a CLB successfully becomes routable through re-pair, it will save a lot of time compared with entering the Route-Each stage.

The re-pair stage presented in Fig. 2 (c) can be formulated as Algorithm 1, which will be invoked after the Route-Once stage fails and before the Route-Each stage. It consists of two main procedures, i.e., candidate LUT selection and candidate LUT reassignment. In addition, we propose an input-equivalent LUT removal technique to improve the overall efficiency of the packing process.

B. Candidate LUT Selection

The first procedure of re-pair is candidate LUT selection (Lines 1-12). The algorithm selects LUTs that cause congestion during intra-routing. These candidate LUTs will

be reassigned to feasible physical LUTs in the subsequent procedure. A candidate LUT should at least meet the following two requirements:

- This LUT causes congestion during intra-routing.
- There are other free physical LUTs in this CLB that can fit this LUT.

For the first requirement, we have recorded the congestion information from the previous failed routing attempt, denoted as C_{prev} in Algorithm 1. However, since each congestion involves at least two LUTs, it is difficult to determine exactly which one causes congestion. Thus, we use each LUT's accumulated number of congestion to evaluate its contribution to the overall congestion. And the algorithm tends to select LUTs that encounter congestion more frequently.

Considering the second requirement, the algorithm tends to select LUTs with fewer inputs, as they offer higher flexibility when searching for alternative physical LUTs. Therefore, in the proposed re-pair algorithm, we use accumulated congestion per input (CPI) for each LUT to evaluate its priority in candidate selection (Line 1), as Equation 1.

$$CPI = \frac{N_{congestion}}{N_{input}} \quad (1)$$

Here, $N_{congestion}$ is the number of times this LUT has been involved in congestion with other LUTs, and N_{input} is the number of inputs of this LUT. When selecting candidate LUTs, the LUT with the highest CPI will be selected (Lines 5-6), and the corresponding BLE will be marked (Line 8). For fairness considerations, the subsequent candidate LUTs will not be selected from these marked BLEs.

In addition, the number of candidates should also be carefully determined. If too many candidates are selected, the exploration space of re-pair will be too big to be efficiently searched. On the contrary, selecting too few LUTs could lead to omissions and result in a failed re-pair attempt. Therefore, in the proposed re-pair stage, the number of candidate LUTs is restricted by the logic size of available physical LUTs, denoted as N_{sel} in Algorithm 1. Specifically, we define the logic size of a k -LUT as 4, a $(k-1)$ -LUT as 2, and a $(k-2)$ -LUT as 1. The value of N_{sel} equals the sum of logic size of unoccupied LUTs in the previous packing solution (Line 2). During the selection procedure, the value of N_{sel} decreases each time an LUT is selected, with the variation determined by the size of the selected LUT (Line 7). The candidate selection ends when N_{sel} is exhausted or all BLEs that encounter congestion are marked (Lines 4, 9-11).

C. Candidate LUT Reassignment

After all candidate LUTs are settled, the algorithm will reassign them back into the CLB, and determine whether a routable packing solution can be obtained (Lines 13-32).

The selected candidate LUTs are stored in a stack, denoted as $CLUT$. These candidate LUTs will be packed back in the reversed order. The algorithm will try pairing each candidate LUT with feasible physical LUTs (Lines 13-15).

To ensure the solution generated by the re-pair stage is always routable, the algorithm verifies its routability (Line 21). However, the routability check is time-consuming. Therefore, when selecting a physical LUT, the logical-physical LUT pair with potentially better intra-routability should be verified first to avoid invoking the intra-routing process as much as possible.

To incorporate routability-awareness into the reassignment procedure, we evaluate the routability through investigating the distribution of logics. Specifically, we use the number of BLE's occupied inputs to estimate its logic density. For a new CLB packing solution generated by the re-pair process, we define its routability score as Equation 2.

$$RS = \sum_{i=1}^{n_{BLE}} input_i^3 \quad (2)$$

Here, $input_i$ is the number of occupied inputs of the i -th BLE in the CLB, and n_{BLE} is the number of BLEs in the CLB. The proposed RS amplifies the penalty for those BLEs that fully use their inputs, which are likely to encounter congestion in routing. A smaller RS indicates better routability for the corresponding packing solution.

In the routability-aware candidate reassignment procedure, the algorithm will select the physical LUT P with the smallest RS (Line 16). After pairing the candidate logical LUT LUT_c with the selected physical LUT P , intra-routing process will be performed to validate whether the current packing solution is routable (Lines 20-21). If routable, the logical-physical LUT pair will be recorded, and the procedure will proceed to handle the next candidate LUT (Lines 22-23). Otherwise, it will proceed to check the next feasible physical LUT. If one of the candidate LUTs cannot find any routable reassignments, the re-pair stage will be terminated immediately, and the Route-Each stage in the original packing algorithm will be invoked (Lines 28-31).

D. Input-Equivalent LUT Removal

As we introduce in Section II, an MLC k -LUT can be used as at most 4 independent $(k-2)$ -LUTs. According to the structure demonstrated in Fig. 1 (d), for each single $(k-2)$ -LUT in the same MLC LUT, its accessible inputs, i.e., the inputs from A_1 to A_{k-2} , are exactly the same. The rest of the inputs, i.e., A_k and A_{k-1} , are used to control the multiplexers for output selection. Thus, these physical $(k-2)$ -LUTs within the same BLE are equivalent from the perspective of input.

Our experimental observation indicates that congestion usually occurs among inputs instead of outputs during the packing process, since the routing resources for outputs are relatively sufficient. Each MLC LUT preserves 4 outputs, as it can store at most 4 logical LUTs. However, when packing, an MLC LUT can hardly be fully occupied by 4 logical LUTs due to the input constraint, where the true bottleneck lies. Thus, when analyzing intra-routability, we only focus on the situation of input occupations.

For these input-equivalent LUTs, if pairing a candidate LUT to one of them does not result in a routable solution, it would

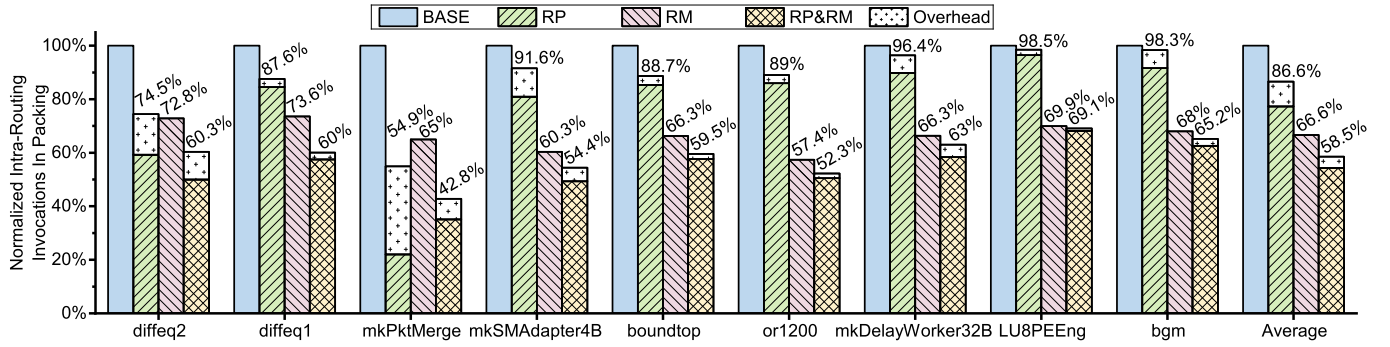


Fig. 3. Normalized intra-routing invocations of different packing methods on various circuits.

also not succeed for the rest of them. Thus, to improve the efficiency of the re-pair process, filtering input-equivalent LUTs from the physical LUT selection is a promising approach.

It should be noted that the input-equivalence only exists among $(k-2)$ -LUTs, as a k -LUT or $(k-1)$ -LUT would not be equivalent to a $(k-2)$ -LUT. When a k -LUT or $(k-1)$ -LUT is configured through combining multiple fractured $(k-2)$ -LUTs, it enables access to the inputs A_k and A_{k-1} . Since these extra inputs offer higher flexibility in pin assignment, the inputs are no longer equivalent to those of $(k-2)$ -LUTs, even in the same BLE.

To identify the input-equivalent physical LUTs, after a packing attempt fails, if the physical LUT being assigned is a $(k-2)$ -LUT, the algorithm will mark the other $(k-2)$ -LUTs within the same BLE as duplicate (Lines 24-26). When a duplicate physical LUT is selected later, the algorithm will skip it and proceed to fetch the next feasible physical LUT (Lines 17-19). Similarly, this technique can also be applied to the Route-Each stage to further enhance the overall packing process efficiency.

IV. EXPERIMENT

In this section, we will evaluate the proposed techniques and report the efficiency improvements in the packing process. Besides, we analyze the overhead and the effects on the quality-of-results (QoR) of the proposed scheme.

A. Experiment Setup

In the evaluation, the proposed techniques are tested on an NVFPGA architecture that uses STT-RAM based MLC LUTs to build up its CLBs. In the experiment, the number of MLC LUT's inputs, i.e., k , is set to 7. This FPGA architecture¹ is based on Altera Stratix IV device.

Our proposed re-pair method is integrated into the widely-used Verilog-To-Route (VTR) CAD flow [25]. It will be activated after the Route-Once stage of the packing process.

We implement several packing methods for comparison.

- **BASE**: It uses the packing algorithm proposed in [26], including two stages, i.e., Route-Once and Route-Each.

¹The architecture file is *k6_frac_N10_mem32k_40nm.xml* (height: 100 width: 100), which has 7500 CLBs and 208 BRAMs in total.

TABLE I
BENCHMARK INFORMATION

Circuit	Number of LUTs				FFs	CLBs
	2/3/4-LUT	5-LUT	6-LUT	7-LUT		
diffeq2	53	57	44	81	96	20
diffeq1	185	26	31	136	193	28
mkPktMerge	172	10	19	15	36	16
mkSM-Adapter4B	580	275	251	608	983	145
boundtop	1362	248	337	678	1671	191
or1200	835	547	376	914	691	231
mkDelay-Worker32B	1771	1227	930	770	2491	394
LU8PEEng	5705	2614	2184	9601	6630	1983
bgm	8068	4415	4662	11923	5362	2931

- **RP (Re-Pair)**: On the basis of **BASE**, it implements the proposed re-pair stage, which is invoked when the Route-Once stage fails.
- **RM (ReMoval)**: On the basis of **BASE**, it adopts the input-equivalent LUT removal technique.
- **RP&RM**: On the basis of **BASE**, it integrates both the re-pair stage and the input-equivalent LUT removal technique in packing.

A series of benchmark circuits, as listed in Table I, are adopted for evaluation, which have different complexities in terms of scale of circuits. The composition of LUT sizes also differs among them. For example, 79.63% of LUTs in *mkPktMerge* have 4 or fewer inputs. On the contrary, 47.76% and 41.02% are 7-LUTs in *LU8PEEng* and *bgm*, respectively.

B. Improvement in Packing Efficiency

With our observation, the packing time is basically proportional to the number of intra-routing invocations, so we use it to evaluate the packing efficiency. Fig. 3 shows the normalized number of intra-routing invocations during the packing process for various circuits. In addition, for methods that involve the re-pair stage, i.e., RP and RP&RM, we specifically highlight the intra-routing invocations of the re-pair stage, i.e., the Overhead in Fig. 3. On average, RP and RM achieve 13.39% and 33.37% packing efficiency improvements, respectively. The result indicates that both the re-pair stage and the input-equivalent LUT removal technique can effectively improve the efficiency of the packing process. These two

techniques are orthogonal and can work together to achieve better improvements. On average, RP&RM achieves 41.48% less intra-routing invocations compared with BASE.

The effectiveness of the re-pair stage is influenced by the circuit's distribution of LUT sizes. Among the circuits we tested, RP achieves the best efficiency improvement in *mkPktMerge*. This circuit contains a lot of CLB's composed of 3-LUTs. These LUTs are gathered to several BLEs while the rest of BLEs are still empty due to the CLB input constraint. Thus, these CLB's can easily become routable through re-pair. On the contrary, for circuits like *LU8PEEng* and *bgm*, which both contain more 7-LUTs, the improvements are less significant. The logical-physical assignments of these 7-LUTs are hard to be adjusted, due to the lack of available physical 7-LUTs.

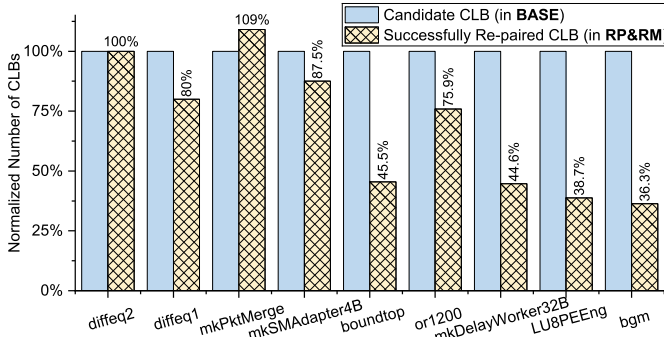


Fig. 4. Normalized number of candidate and successfully re-paired CLB's.

Fig. 4 compares the number of successfully re-paired CLB's (in RP&RM) and candidate CLB's (in BASE), i.e., CLB's that contain same LUTs in both the Route-Once stage and the Route-Each stage. On average, the proposed method successfully handles 68.62% of the candidate CLB's, which verifies the effectiveness of the proposed routability-aware re-pair technique. It should be noted that a successfully re-paired CLB may generate a different packing solution from the one in BASE, which changes the follow-up packing process so the subsequent CLB's are different. Thus, it is possible for the number of successfully re-paired CLB's to exceed the candidate CLB's, as observed in *mkPktMerge*.

The experiment results show that the proposed re-pair stage can effectively identify the re-pairable CLB's and make them routable with less intra-routing invocations.

C. Overhead Analysis

The proposed re-pair stage also involves intra-routing invocations, which are referred to as overhead. As shown in Fig. 3, on average, the re-pair stage contributes 4.15% of the total number of intra-routing invocations. However, the original packing process needs $4.45\times$ more intra-routing invocations to pack these CLB's through the Route-Each stage. Thus, the re-pair stage delivers significant packing time reduction with little overhead.

To reduce the overhead, we propose *RS* to guide the reassignment procedure, and define LUT input-equivalence to filter unnecessary intra-routing attempts. As Fig. 5 shows, the

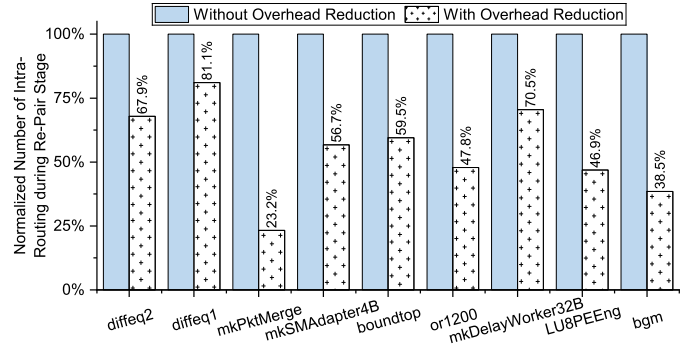


Fig. 5. Normalized number of intra-routing invocations in re-pair stage.

TABLE II
THE AREA AND PERFORMANCE EVALUATION.

Circuit	Area(#CLBs)			Performance(ns)		
	BASE	RP&RM	Ratio	BASE	RP&RM	Ratio
diffeq2	20	20	100%	16.41	16.34	99.58%
diffeq1	28	27	96.43%	21.15	21.32	100.8%
mkPktMerge	16	16	100%	4.041	3.980	98.50%
mkSM-Adapter4B	145	144	99.31%	5.898	5.743	97.38%
boundtop	192	191	99.48%	6.472	6.169	95.31%
or1200	231	230	99.57%	12.68	12.57	99.13%
mkDelay-Worker32B	394	393	99.75%	8.080	7.455	92.27%
LU8PEEng	1990	1979	99.45%	103.4	103.2	99.76%
bgm	2927	2926	99.97%	24.55	24.97	101.7%
Average	660	658	99.33%	22.53	22.42	98.27%

above-mentioned overhead reduction techniques achieve an average reduction of overhead by 54.69%.

D. Design Quality Evaluation

Since the re-pair stage potentially generates a different packing solution, the final design may be different in area and performance. Table II shows the QoR of BASE and RP&RM, where area is estimated using the number of CLB's, and the performance is evaluated using the delay of the critical path. The proposed method improves the area by 0.67% and the performance by 1.73%, on average. The results indicate that, the proposed techniques would not degrade the QoR, but lead to slight QoR improvement.

V. CONCLUSION

In this paper, we propose a routability-aware re-pair strategy to improve the efficiency of the packing process in the CAD flow for MLC NVFPGAs. To reduce the intra-routing invocations, we propose the re-pair stage, which contains two procedures, i.e., candidate LUT selection and candidate LUT reassignment. To incorporate the routability-awareness into the re-pair stage, we propose a routability evaluation technique to guide the candidate LUT reassignment. In addition, leveraging the input equivalence of some physical LUTs, we identify and remove the unnecessary intra-routing attempts during packing, thereby further improving the overall efficiency. We conduct experiments to evaluate the efficiency improvements of the proposed techniques and give discussions. The experimental results show an average packing time reduction of 41.48%.

REFERENCES

- [1] G. Singh, M. Alser, D. S. Cali, D. Diamantopoulos, J. Gómez-Luna, H. Corporaal, and O. Mutlu, "FPGA-based near-memory acceleration of modern data-intensive applications," *International Symposium on Microarchitecture (MICRO)*, vol. 41, no. 4, pp. 39–48, 2021.
- [2] H. Zhang, J. Zuo, H. Zheng, S. Liu, M. Luo, and M. Zhao, "Implementing neural networks on nonvolatile FPGAs with reprogramming," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 43, no. 11, pp. 3961–3972, 2024.
- [3] K. Zeng, Q. Ma, J. W. Wu, Z. Chen, T. Shen, and C. Yan, "FPGA-based accelerator for object detection: a comprehensive survey," *The Journal of Supercomputing (TJSC)*, vol. 78, no. 12, pp. 14 096–14 136, 2022.
- [4] N. Zhang, X. Wei, H. Chen, and W. Liu, "FPGA implementation for CNN-based optical remote sensing object detection," *MDPI Electronics*, vol. 10, no. 3, p. 282, 2021.
- [5] Y. Zhu, X. Wang, L. Ju, and S. Guo, "FxFHNN: FPGA-based acceleration framework for homomorphic encrypted CNN inference," in *International Symposium on High Performance Computer Architecture (HPCA)*, 2023, pp. 896–907.
- [6] A. A. Yazdeen, S. R. Zeebaree, M. M. Sadeeq, S. F. Kak, O. M. Ahmed, and R. R. Zebari, "FPGA implementations for data encryption and decryption via concurrent and parallel computation: A review," *Qubahan Academic Journal*, vol. 1, no. 2, pp. 8–16, 2021.
- [7] H. Chen, J. Zhang, Y. Du, S. Xiang, Z. Yue, N. Zhang, Y. Cai, and Z. Zhang, "Understanding the potential of FPGA-based spatial acceleration for large language model inference," *ACM Transactions on Reconfigurable Technology and Systems (TRETS)*, 2024.
- [8] S. Zeng, J. Liu, G. Dai, X. Yang, T. Fu, H. Wang, W. Ma, H. Sun, S. Li, Z. Huang *et al.*, "Flightllm: Efficient large language model inference with a complete mapping flow on FPGAs," in *Symposium on Field Programmable Gate Arrays (FPGA)*, 2024, pp. 223–234.
- [9] H. Sharma, J. Park, D. Mahajan, E. Amaro, J. K. Kim, C. Shao, A. Mishra, and H. Esmailzadeh, "From high-level deep neural models to FPGAs," in *International Symposium on Microarchitecture (MICRO)*, 2016, pp. 17:1–17:12.
- [10] A. Boutros, E. Nurvitadhi, R. Ma, S. Gribok, Z. Zhao, J. C. Hoe, V. Betz, and M. Langhammer, "Beyond peak performance: Comparing the real performance of AI-optimized FPGAs and GPUs," in *International Conference on Field-Programmable Technology (FPT)*, 2020, pp. 10–19.
- [11] A. Tauhid, L. Xu, M. Rahman, and E. Tomai, "A survey on security analysis of machine learning-oriented hardware and software intellectual property," *Elsevier High-Confidence Computing (HCC)*, pp. 100–114, 2023.
- [12] A. Putnam, A. M. Caulfield, E. S. Chung, D. Chiou, K. Constantinides, J. Demme, H. Esmailzadeh, J. Fowers, G. P. Gopal, J. Gray, M. Haselman, S. Hauck, S. Heil, A. Hormati, J. Kim, S. Lanka, J. R. Larus, E. Peterson, S. Pope, A. Smith, J. Thong, P. Y. Xiao, and D. Burger, "A reconfigurable fabric for accelerating large-scale datacenter services," *Communications of the ACM (CACM)*, vol. 59, no. 11, pp. 114–122, 2016.
- [13] Y. Chen, J. Zhao, and Y. Xie, "3D-nonFAR: three-dimensional non-volatile FPGA architecture using phase change memory," in *International Symposium on Low Power Electronics and Design (ISLPED)*, 2010, pp. 55–60.
- [14] K. Huang, Y. Ha, R. Zhao, A. Kumar, and Y. Lian, "A low active leakage and high reliability phase change memory (PCM) based non-volatile FPGA storage element," *IEEE Transactions on Circuits and Systems I: Regular Papers (TCASI)*, vol. 61, no. 9, pp. 2605–2613, 2014.
- [15] P. Narayanan, S. Ambrogio, A. Okazaki, K. Hosokawa, H. Tsai, A. Nomura, T. Yasuda, C. Mackin, S. C. Lewis, A. Friz *et al.*, "Fully on-chip MAC at 14 nm enabled by accurate row-wise programming of PCM-based weights and parallel vector-transport in duration-format," *IEEE Transactions on Electron Devices (TED)*, vol. 68, no. 12, pp. 6629–6636, 2021.
- [16] S. Paul, S. Mukhopadhyay, and S. Bhunia, "Hybrid CMOS-STTRAM non-volatile FPGA: design challenges and optimization approaches," in *International Conference on Computer Aided Design (ICCAD)*, 2008, pp. 589–592.
- [17] B. Khaleghi and H. Asadi, "A resistive RAM-based FPGA architecture equipped with efficient programming circuitry," *IEEE Transactions on Circuits and Systems I: Regular Papers (TCASI)*, vol. 65, no. 7, pp. 2196–2209, 2018.
- [18] Y. Long, T. Na, and S. Mukhopadhyay, "ReRAM-based processing-in-memory architecture for recurrent neural network acceleration," *IEEE Transactions on Very Large Scale Integration Systems (TVLSI)*, vol. 26, no. 12, pp. 2781–2794, 2018.
- [19] H. P. Wong, H. Lee, S. Yu, Y. Chen, Y. Wu, P. Chen, B. Lee, F. T. Chen, and M. Tsai, "Metal-oxide RRAM," *Proceedings of the IEEE*, vol. 100, no. 6, pp. 1951–1970, 2012.
- [20] K. Liu, M. Zhao, L. Ju, Z. Jia, C. J. Xue, and J. Hu, "Design exploration for multiple level cell based non-volatile FPGAs," in *International Conference on Computer Design (ICCD)*, 2017, pp. 257–264.
- [21] J. Cong and B. Xiao, "FPGA-RPI: A novel FPGA architecture with RRAM-based programmable interconnects," *IEEE Transactions on Very Large Scale Integration Systems (TVLSI)*, vol. 22, no. 4, pp. 864–877, 2013.
- [22] D. Choi, K. Choi, and J. D. Villasenor, "New non-volatile memory structures for FPGA architectures," *IEEE Transactions on Very Large Scale Integration Systems (TVLSI)*, vol. 16, no. 7, pp. 874–881, 2008.
- [23] H. Ochi, K. Yamaguchi, T. Fujimoto, J. Hotate, T. Kishimoto, T. Higashi, T. Imagawa, R. Doi, M. Tada, T. Sugibayashi *et al.*, "Via-switch FPGA: Highly dense mixed-grained reconfigurable architecture with overlay via-switch crossbars," *IEEE Transactions on Very Large Scale Integration Systems (TVLSI)*, vol. 26, no. 12, pp. 2723–2736, 2018.
- [24] L. Ju, X. Sui, S. Li, M. Zhao, C. J. Xue, J. Hu, and Z. Jia, "NVM-based FPGA block RAM with adaptive SLC-MLC conversion," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 37, no. 11, pp. 2661–2672, 2018.
- [25] K. E. Murray, O. Petelin, S. Zhong, J. M. Wang, M. Eldafrawy, J. Legault, E. Sha, A. Graham, J. Wu, M. J. P. Walker, H. Zeng, P. Patros, J. Luu, K. B. Kent, and V. Betz, "VTR 8: High-performance CAD and customizable FPGA architecture modelling," *ACM Transactions on Reconfigurable Technology and Systems (TRETS)*, vol. 13, no. 2, pp. 9:1–9:55, 2020.
- [26] K. Liu, M. Zhao, L. Ju, Z. Jia, J. Hu, and C. J. Xue, "Applying multiple level cell to non-volatile FPGAs," *ACM Transactions on Embedded Computing Systems (TECS)*, vol. 19, no. 4, pp. 27:1–27:22, 2020.
- [27] W. Li, S. Dhar, and D. Z. Pan, "UTPlaceF: A routability-driven FPGA placer with physical and congestion aware packing," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 37, no. 4, pp. 869–882, 2018.
- [28] M. A. Elgammal and V. Betz, "Breaking boundaries: Optimizing FPGA CAD with flexible and multi-threaded re-clustering," in *International Symposium on Highly Efficient Accelerators and Reconfigurable Technologies (HEART)*, 2023, pp. 11–18.
- [29] L. Yu, B. Guo, T. Zhi, and L. Bai, "Improving seed-based FPGA packing with indirect connection for realization of neural networks," *MDPI Electronics*, vol. 12, no. 12, p. 2691, 2023.
- [30] Q. Chen, M. Shen, and N. Xiao, "DP-Pack: Distributed parallel packing for FPGAs," in *International Conference on Field-Programmable Technology (FPT)*, 2018, pp. 282–285.
- [31] R. S. Rajarathnam, Z. Jiang, M. A. Iyer, and D. Z. Pan, "DREAMPlaceFPGA-PL: An open-source GPU-accelerated packer-legalizer for heterogeneous FPGAs," in *International Symposium on Physical Design (ISPD)*, 2023, pp. 175–184.